

OpenCode Cookbook

从安装、上下文与权限，到 Agent / Skill / MCP / Tool / Server 的工程化实践

定位

这不是命令速查表，而是一份可复用的 Coding Agent Runtime 操作手册。每个核心机制都回答两件事：为什么这样设计（know-why），以及如何稳定落地（know-how）。

资料核验日期：2026-08-13

适用：当前 anomalyco/opencode 主线文档与 1.x 系列

如何使用这本 Cookbook

如果你第一次使用 OpenCode：按第 1 → 6 → 7 → 8 → 16 章走一遍。若你正在研究 Coding Agent 架构：优先阅读第 2、7、8、9、11、12、15、18、23 章。

路径	建议章节	目标
30 分钟上手	1, 3, 4, 5, 6	安装、连接模型、初始化仓库、完成一次修改
团队工程化	7, 8, 9, 10, 16	规则、权限、Agent/Skill/Command 模板化
扩展能力	11, 12, 13, 17, 18	Custom Tool、MCP、Plugin、Server/SDK
研究 Agent Runtime	2, 7, 8, 15, 18, 23	理解 Harness、Context、Action Space、Lifecycle
排错与稳定性	14, 15, 22	LSP/验证、Context、常见故障

章节地图

- 1. 30 分钟 Quick Start
- 2. 心智模型：OpenCode 到底是什么
- 3. 安装与升级
- 4. Provider 与模型配置
- 5. /init 与 AGENTS.md
- 6. TUI 日常操作
- 7. Permission：安全与 Action Space
- 8. Agent / Subagent
- 9. Skills：渐进式披露
- 10. Commands：把 Prompt 产品化
- 11. Custom Tools：把能力接入 Runtime
- 12. MCP：连接外部工具系统
- 13. Plugins：扩展生命周期
- 14. LSP / Formatter / 验证链
- 15. Context Engineering 与 Compaction
- 16. 推荐项目目录与基础配置
- 17. References：跨仓库上下文
- 18. CLI / Server / SDK / ACP
- 19. GitHub Automation
- 20. 12 个高频 Recipe
- 21. 团队治理与评测
- 22. Troubleshooting
- 23. 可沉淀的 know-how
- 附录 A. 配置模板
- 附录 B. Agent / Skill / Command 模板
- 附录 C. 命令速查
- 附录 D. 官方资料索引

1. 30 分钟 Quick Start

目标

从“机器上没有 OpenCode”到“在一个 Git 仓库中完成：理解 → 计划 → 修改 → 测试 → diff review”。

Step 1 - 安装

```
# macOS / Linux
curl -fsSL https://opencode.ai/install | bash

# macOS / Linux (Homebrew, 官方 README 推荐 tap)
brew install anomalyco/tap/opencode

# Node package manager
npm i -g opencode-ai@latest

# 验证
opencode --version
```

Step 2 - 进入项目并启动

```
cd /path/to/your-project
opencode
```

Step 3 - 连接模型

```
/connect
/models
```

Step 4 - 初始化仓库规则

```
/init
```

OpenCode 会扫描仓库并创建或更新项目根目录中的 AGENTS.md。官方建议将项目级 AGENTS.md 提交到 Git，以让未来 session 共享稳定的仓库知识。

Step 5 - 先计划，后修改

```
# 切到 Plan (Tab)，然后输入：
Investigate how authentication works in this repository.
Identify the main execution path, relevant files, tests, and the smallest safe change.
Do not edit files yet.

# 看完计划后切到 Build，再输入：
Implement the agreed change.
Run focused tests, typecheck, inspect git diff, and summarize verification.
```

Step 6 - 自己检查结果

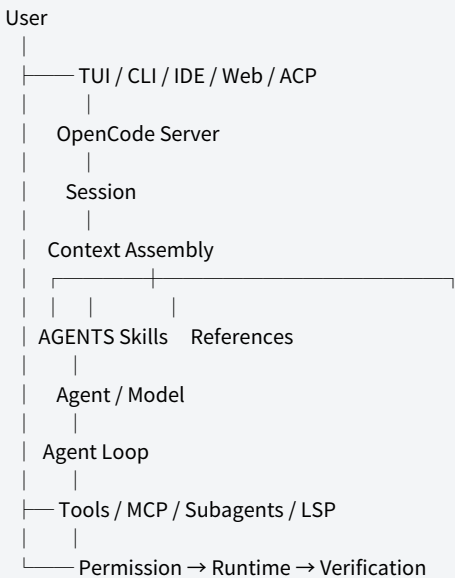
```
!git status
!git diff
```

成功标准

不是“模型说 done”，而是代码改动可解释、diff 可审、测试/类型检查通过、没有意外副作用。

2. 心智模型：OpenCode 到底是什么

把 OpenCode 只理解成“终端聊天工具”会错过它最有价值的部分。更准确的模型是：一个可配置的 Coding Agent Runtime，加上 TUI/CLI/IDE/HTTP/SDK 等客户端与接入层。



2.1 五个关键层

层	职责	工程问题
Client	TUI、CLI、IDE、ACP、Web	人/系统如何发起任务？
Context	规则、历史、Skills、References	模型此刻应该知道什么？
Reasoning	Agent + Model	谁来做、用什么模型、做到几步？
Capability	Built-in tools、Custom Tools、MCP	模型能采取什么动作？
Governance	Permission、Git、tests、CI	哪些动作允许？如何验证/回滚？

核心 know-why

Agent 的质量不是模型参数单变量。Coding Agent = Model × Context × Tool affordance × Permission × Runtime × Verification。模型很强，但上下文和动作空间设计差，仍会稳定地产生差结果。

2.2 Server-first 架构

官方 Server 文档说明：运行 opencode 时会启动 TUI 与 Server，TUI 是 Server 的客户端；独立的 `opencode serve` 会暴露 OpenAPI 3.1 接口。这个设计让同一个 runtime 可以被 TUI、IDE、SDK 或你自己的自动化复用。

工程意义

将“交互界面”与“Agent Runtime”解耦后，评测、CI、批处理、IDE 集成都不需要重新实现 Agent loop。

3. 安装、升级与环境卫生

3.1 推荐安装顺序

环境	推荐方式	理由
macOS/Linux	anomalyco/tap/opencode	官方 README 标注为推荐且更新更及时
Node 开发环境	npm i -g opencode-ai@latest	团队已统一 Node 工具链时方便
Windows	Scoop / Chocolatey / npm；复杂 Unix workflow 可考虑 WSL	尽量减少 shell/路径差异
隔离实验	Docker / 临时用户 / VM	注意 OpenCode 本身不提供沙箱

3.2 升级前检查

- 记录当前 `opencode --version`。
- 将 `opencode.json(c)`、`.opencode/`、`AGENTS.md` 纳入版本控制。
- 实验性配置放独立分支；升级后先用小仓库跑 smoke test。

3.3 安全边界

重要

OpenCode 官方 Security 文档明确说明 Agent 不在沙箱中运行。Permission 是重要的授权/可见性机制，但不能替代 OS/容器/VM 隔离。对于未知仓库、第三方脚本、具有外部副作用的工具，应使用额外隔离。

4. Provider 与模型配置

OpenCode 使用 AI SDK / Models.dev 体系支持大量模型提供商，并支持 OpenAI-compatible 自定义端点与本地模型。连接流程与运行配置是两层：凭据负责“能认证”，provider/model 配置负责“如何调用”。

4.1 常规连接

```
/connect
/models
opencode auth list
```

4.2 自定义 OpenAI-compatible Provider

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "myprovider": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "My Provider",
      "options": {
        "baseUrl": "https://api.example.com/v1"
      },
      "models": {
        "my-model": { "name": "My Model" }
      }
    }
  },
  "model": "myprovider/my-model"
}
```

4.3 密钥管理

```
export MY_API_KEY="..."

{
  "provider": {
    "myprovider": {
      "options": { "apiKey": "{env:MY_API_KEY}" }
    }
  }
}
```

know-how

把“认证信息”与“项目行为配置”分离。不要把密钥写进可提交的`opencode.json`。对团队项目，配置文件可入库，凭据走环境变量或OpenCode的credential storage。

4.4 模型路由策略

任务	偏好模型特征	Agent 配置建议
仓库探索	快、便宜、工具调用稳定	Explore/Scout 使用快模型
计划/设计	推理稳定、长上下文	Plan 低温度
复杂实现	代码能力、工具调用、长回合	Build 使用主力模型
Code review	准确、保守	Reviewer 低温度 + 禁止 edit

5. /init 与 AGENTS.md：把仓库知识变成长期上下文

官方 `/init` 的目标不是生成项目百科，而是提炼未来 Agent session 高频需要的信息：build/lint/test 命令、验证顺序、架构结构、项目约定、setup quirks、operational gotchas，以及已有规则来源。

5.1 为什么 AGENTS.md 有用

没有 AGENTS.md：
每个 session 都要重新猜 → 如何测试？目录怎么分？有哪些禁忌？

有 AGENTS.md：
Repository facts → stable context contract → less rediscovery

5.2 推荐模板

```
# Repository

TypeScript monorepo using pnpm.

## Structure
- packages/api: backend
- packages/web: frontend
- packages/shared: shared contracts

## Commands
- Install: `pnpm install`
- Focused test: `pnpm vitest <file>`
- Typecheck: `pnpm typecheck`
- Lint: `pnpm lint`

## Workflow
After changing TypeScript:
1. run focused tests
2. run typecheck
3. inspect `git diff`

## Conventions
- Reuse existing abstractions before adding dependencies.
- Keep public APIs backward compatible unless explicitly requested.
- Do not edit generated files directly.

## References
- docs/architecture.md
- CONTRIBUTING.md
```

5.3 什么不该放进去

- 一次性任务详情：应留在当前 session/issue。
- 很长的 API 文档：更适合 References / Skill / MCP。
- 低频流程：更适合 Skill。
- 可以从代码瞬间读出的事实：不要重复。
- 已过期的版本/人员信息：除非有维护机制。

判定规则

AGENTS.md 只放“高频 + 稳定 + 全局有用”。Skill 放“按需 + 程序化流程”。Reference 放“体量大 + 需要查阅”的资料。

6. TUI 日常操作

动作	用途	推荐场景
@file / @reference	显式加入文件/引用	让问题边界可见，减少盲搜
!command	执行 shell 并把结果带回 session	git status/diff、快速诊断
/compact	压缩长 session；别名 /summarize	上下文接近满或历史噪声大
/undo	撤销最近消息及相关文件改动	试错失败；项目需 Git
/redo	恢复撤销	在两个方案间往返
/details	展开工具执行细节	审计 agent 行为

6.1 高信息密度 Prompt

Review @src/auth/service.ts and @src/auth/middleware.ts.

Goal: identify why refresh tokens occasionally fail.

Constraints:

- do not edit files
- prefer existing abstractions
- separate evidence from hypotheses

Return:

1. execution path
2. likely root cause
3. affected tests
4. smallest fix

know-why

好的 Prompt 不是越长越好，而是主动降低搜索空间：目标、边界、证据要求、输出结构。

7. Permission：安全机制，也是 Action Space 设计

Permission 的结果是`allow` / `ask` / `deny`。官方当前默认允许操作；工程项目通常应显式配置高风险动作。规则按模式匹配，最后一个命中的规则获胜，因此常见写法是先`\*`，再更具体的规则。

7.1 安全基线

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "**": "ask",
    "read": "allow",
    "glob": "allow",
    "grep": "allow",
    "lsp": "allow",
    "skill": "allow",
    "edit": "ask",
    "bash": {
      "**": "ask",
      "git status*": "allow",
      "git diff*": "allow",
      "git log*": "allow",
      "git push*": "deny"
    }
  }
}
```

7.2 风险分层

动作类别	例子	推荐默认
只读/可观察	read, grep, git status, git diff	allow
本地可逆修改	edit, format	ask → 稳定后可 selective allow
外部副作用	push, deploy, create remote issue	ask 或 deny
高破坏/不可逆	危险删除、修改关键环境	deny + OS 隔离

7.3 Auto mode

```
opencode --auto
opencode run --auto "Refactor this module"
```

官方语义：Auto 只自动批准原本会 `ask` 的请求；显式 `deny` 仍然生效。正确做法是先把硬边界写成 deny，再考虑 auto，而不是把所有权限放开。

关键 know-how

Permission 不只是“弹不弹确认框”。被 deny 的能力会缩小 Agent 可用动作空间，减少模型在无效/危险选项上的探索。

7.4 Approval fatigue

如果所有读取、grep、git status 都要确认，用户很快形成机械点击。更稳的策略是：低风险动作默认放行，高风险/外部副作用保留 ask，明确危险动作 deny。

8. Agent / Subagent : 角色化 + Context Isolation

当前内置 primary agents 包括 Build 与 Plan；subagents 包括 General、Explore、Scout。Plan 默认限制修改并对 bash 请求批准；Explore 是快速只读代码探索；Scout 偏外部文档与依赖研究。

Agent	类型	典型用途
Build	primary	实现、测试、系统命令
Plan	primary	分析、设计、变更计划
General	subagent	复杂多步研究/并行工作
Explore	subagent	只读代码搜索
Scout	subagent	依赖源码、外部文档/上游实现

8.1 自定义 Reviewer

```
---
description: Review changes for correctness, regressions and maintainability
mode: subagent
temperature: 0.1
permission:
  edit: deny
  bash:
    "**": ask
    "git diff*": allow
    "git log*": allow
---

Review the implementation.
Focus on correctness, edge cases, security, duplicated logic, and missing tests.
Do not modify files.
```

放置路径：项目 ``.opencode/agents/reviewer.md``；全局 ``~/config/opencode/agents/``。文件名即 Agent 名。

8.2 Task permissions

```
{
  "agent": {
    "orchestrator": {
      "mode": "primary",
      "description": "Routes work to specialized subagents",
      "permission": {
        "task": {
          "**": "deny",
          "explore": "allow",
          "reviewer": "ask"
        }
      }
    }
  }
}
```

为什么 Subagent 有价值

真正高价值的不仅是并行，而是 Context Isolation：大量探索日志、依赖研究、中间失败可以留在 child session，只把关键结论带回主上下文。

8.3 模型路由

Primary agent 没指定模型时使用全局模型；subagent 没指定模型时通常继承调用它的 primary 模型。可以对 Explore/Reviewer 单独指定更快或更保守的模型。

9. Skills：把组织 know-how 做成按需加载流程

Skill 是 `SKILL.md`。OpenCode 会在项目、全局，以及 Claude/Agent-compatible 目录中发现 skills。Agent 先看到 skill 的 name/description，需要时通过 `skill` tool 加载完整内容。

```
startup context
|
├─ skill: database-migration — Safely create schema migrations
├─ skill: release — Prepare releases
└─ skill: incident-debug — Diagnose production incidents
    |
    agent decides relevant
    |
    skill({name: ...})
    |
    load full SKILL.md
```

9.1 推荐目录

```
.opencode/skills/
├─ database-migration/
│   └─ SKILL.md
├─ release/
│   └─ SKILL.md
└─ api-change/
    └─ SKILL.md
```

9.2 Skill 模板

```

---
name: database-migration
description: Safely create and validate database schema migrations
---

# Goal
Create a backwards-compatible migration with a verified rollback path.

## Procedure
1. Inspect current schema and prior migrations.
2. Identify compatibility risks.
3. Generate the migration using repository tooling.
4. Review generated SQL.
5. Run migration tests.
6. Verify rollback or forward-fix strategy.

## Verification
- focused migration test
- typecheck
- schema diff

## Guardrails
- never edit production data
- never bypass repository migration tooling

```

9.3 描述是路由器

Tool-selection know-how

`description` 不是文案装饰，而是 Agent 判断“何时加载这个 Skill”的检索键。描述应包含任务触发条件与差异化语义，避免多个 Skill 都写成模糊的“helps with development”。

10. Commands：把高频 Prompt 产品化

Command 适合“用户明确触发”的高频入口，例如 `/review`、`/investigate`、`/test-changed`。它可以注入参数、shell 输出、文件引用，也可以指定 agent/model/subtask。

10.1 Review Command

```
---
description: Review current code changes
agent: plan
---
```

Review the current changes.

Git diff:
!`git diff`

Focus on:

- correctness
- regression risk
- security
- missing tests

Do not modify files.

保存为 `.opencode/commands/review.md`，运行 `/review`。

10.2 参数

```
---
description: Investigate a target module
---
Investigate $1.
Goal: $ARGUMENTS
Return evidence, root cause hypotheses, and verification steps.
```

官方支持 `\$ARGUMENTS` 与 `\$1`、`\$2` 等位置参数。

10.3 Subtask

上下文隔离技巧

对“生成大报告、扫描大量文件、依赖研究”等 command 设置 `subtask: true`，让任务进入 subagent/child session，避免主会话被大量中间结果污染。

11. Custom Tools：把项目能力直接接入 Runtime

Custom Tool 适合本项目独有能力：查询本地开发数据库、读取构建产物、检查 schema、调用内部脚本。Tool 定义使用 TypeScript/JavaScript，但内部可调用 Python、shell 等任意实现。

11.1 最小 Tool

```
import { tool } from "@opencode-ai/plugin"

export default tool({
  description: "Read project build metadata",
  args: {
    target: tool.schema.string().describe("Build target to inspect")
  },
  async execute(args, context) {
    return `target=${args.target}; worktree=${context.worktree}`
  }
})
```

文件放在 ``.opencode/tools/build-info.ts``，文件名成为 tool 名。Tool context 可提供 agent、sessionID、messageID、directory、worktree 等运行态信息。

11.2 Schema = API + Prompt

差设计	好设计
<code>`query: string`</code>	<code>`query: Read-only SQL used to inspect the local dev database`</code>
<code>`path: string`</code>	<code>`path: Repository-relative source file to analyze`</code>
<code>`mode: string`</code>	枚举 + 明确每个值的语义

核心 know-how

LLM 选工具时能看到的主要就是 Tool 名、description 与参数 schema。传统 API 只追求调用者“能调用”；Agent Tool 还要追求概率模型“能选对、能填对、能理解返回值”。

11.3 返回值设计

- 返回下一步决策需要的字段，不要倾倒整个数据库对象。
- 错误信息应给出可行动的修复方向。
- 稳定结构优于随意自然语言。
- 数据量大时先过滤/聚合，再把高价值结果交给模型。

12. MCP：把外部能力接入 Agent

OpenCode 支持 local 与 remote MCP。加入后，MCP tools 会和内置 tools 一起供 LLM 使用。官方特别提醒：MCP 工具定义会增加上下文，启用太多服务器会迅速消耗 token。

12.1 Local MCP

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "my-local": {
      "type": "local",
      "command": ["npx", "-y", "my-mcp-server"],
      "enabled": true
    }
  }
}
```

12.2 Remote MCP

```
{
  "mcp": {
    "company-tools": {
      "type": "remote",
      "url": "https://mcp.example.com/mcp",
      "headers": {
        "Authorization": "Bearer {env:MY_API_KEY}"
      }
    }
  }
}
```

12.3 OAuth 管理

```
opencode mcp list
opencode mcp auth company-tools
opencode mcp logout company-tools
opencode mcp debug company-tools
```

12.4 MCP vs Custom Tool

判断	Custom Tool	MCP
作用域	当前项目/OpenCode 工作流	跨 Agent/客户端/项目
实现成本	低，直接贴近 repo	需要 MCP server
适合	本地脚本、内部项目语义	GitHub/Jira/数据库服务/企业工具
Context 成本	通常较可控	工具多时可能显著增加

MCP 最佳实践

不要把“能接入”误认为“应该常开”。工具 schema 也占上下文。按工作流启用最小 MCP 集合，并通过 Permission/Agent 进一步限制暴露。

13. Plugins : 扩展 OpenCode 生命周期

Plugin 是 JS/TS 模块，返回 hooks 对象。它比 Custom Tool 更高一层：不仅增加动作，还可以观察/修改 runtime 生命周期，例如通知、环境注入、日志、上下文 compaction 定制等。

```
.opencode/plugins/
└── safety.ts

~/config/opencode/plugins/
└── telemetry.ts
```

13.1 选择边界

需求	优先机制
让模型多一个动作	Custom Tool / MCP
定义操作流程	Skill
增加快捷入口	Command
改变生命周期/事件处理	Plugin
改变模型角色和权限	Agent

设计原则

不要用 Plugin 解决 Skill 能解决的问题。越靠近 Runtime 底层，维护/回归成本越高。

14. LSP / Formatter / Verification : 让“理解”闭环到“可运行”

OpenCode 可以配置 LSP。当前主线文档中，`lsp: true` 用于启用内置 LSP 集成；也可以对单个 server 配置 command、extensions、env、initialization。

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true
}
```

14.1 LSP 与测试不是替代关系

```
grep / read → lexical evidence
LSP      → semantic diagnostics / symbols
compiler → language correctness
typecheck → type correctness
unit tests → local behavior
integration → interaction behavior
git diff → unintended change review
```


验证铁律

Agent “读懂了” 不是完成条件；静态工具 “无报错” 也不是完成条件。对 coding task，验证链至少应包含与改动匹配的 compiler/typecheck/test/diff。

15. Context Engineering 与 Compaction

Agent 的 Context Window 是工作内存，不是无限仓库。OpenCode 的 rules、skills、references、tool schemas、MCP、历史消息、tool results 都会竞争上下文预算。

15.1 三类 Context

类型	机制	设计目标
长期稳定	AGENTS.md / instructions	每次都值得带入
按需知识	Skill / Reference	相关时加载/查阅
运行历史	messages / tool results	保留决策，压缩噪声

15.2 Compaction

```
{
  "compaction": {
    "auto": true,
    "prune": true,
    "reserved": 10000
  }
}
```

官方配置中 `auto` 控制接近上下文限制时自动压缩，`prune` 可删除旧 tool outputs，`reserved` 为压缩预留 token 缓冲。TUI 还可以主动执行 `/compact`。

15.3 什么时候手动 compact

- 一次长调试已经找到根因，后面只剩实现。
- 大量 grep/read 输出已经失去价值。
- 切换任务阶段：探索 → 实现、实现 → review。
- 模型开始重复搜索、忘记最近决策或明显受旧假设干扰。

高质量压缩应该保留

目标、已确认事实、关键决策、约束、改过哪些文件、验证状态、尚未解决的问题。应丢弃：重复日志、过期假设、大段中间 tool output。

16. 推荐项目目录与基础配置

```

my-project/
├── AGENTS.md
├── opencode.jsonc
├── .opencode/
│   ├── agents/
│   │   ├── reviewer.md
│   │   └── tester.md
│   ├── commands/
│   │   ├── review.md
│   │   └── investigate.md
│   ├── skills/
│   │   ├── database-migration/SKILL.md
│   │   └── release/SKILL.md
│   ├── tools/
│   │   └── project-info.ts
│   └── plugins/
│       └── safety.ts
├── src/
├── tests/
└── docs/

```

16.1 基础 opencode.jsonc

```

{
  "$schema": "https://opencode.ai/config.json",

  "lsp": true,

  "permission": {
    "**": "ask",
    "read": "allow",
    "glob": "allow",
    "grep": "allow",
    "lsp": "allow",
    "skill": "allow",
    "edit": "ask",
    "bash": {
      "**": "ask",
      "git status*": "allow",
      "git diff*": "allow",
      "git log*": "allow",
      "git push*": "deny"
    }
  },

  "compaction": {
    "auto": true,
    "prune": true,
    "reserved": 10000
  },

  "watcher": {
    "ignore": ["node_modules/**", "dist/**", ".git/**"]
  }
}

```

说明

这是一份保守工程基线，不是官方唯一推荐。核心思想是：默认 ask，显式 allow 低风险观察动作，deny 明确外部高风险动作；团队可逐步放宽。

17. References：多仓库 / 大文档上下文

References 可以把当前项目外的本地目录或 Git 仓库作为可访问引用。适合产品文档、共享库、SDK、上游源码、历史实现。

```
{
  "references": {
    "docs": {
      "path": "../product-docs",
      "description": "Use for product behavior and documentation conventions"
    },
    "sdk": {
      "repository": "anomalyco/opencode-sdk-js",
      "branch": "main",
      "description": "Use for SDK implementation details"
    }
  }
}
```

使用：`Compare this implementation with @sdk/src/client.ts`。带 description 的 references 会告诉 Agent “何时使用”；没有 description 的仍可由用户通过 @ autocomplete 主动引用。

know-how

Reference 的价值不是把另一个仓库永久塞进 Context，而是给 Agent 一个“可发现、可按需读取”的外部工作区。

18. CLI / Server / SDK / ACP：把 OpenCode 从工具变成平台

18.1 CLI 直接运行

```
opencode run "Explain the architecture"
opencode run --agent plan "Review the current git diff"
opencode run --continue "Continue from the previous session"
```

CLI 适合脚本、CI、评测 harness；TUI 适合人机协作。

18.2 Headless Server

```
OPENCODE_SERVER_PASSWORD="strong-secret" opencode serve --port 4096

# OpenAPI 3.1
# http://localhost:4096/doc
```

默认监听 127.0.0.1:4096。若要暴露到网络，必须配置认证、网络边界与 CORS；不要把未保护的 agent runtime 直接暴露到公网。

18.3 SDK

```
npm install @opencode-ai/sdk

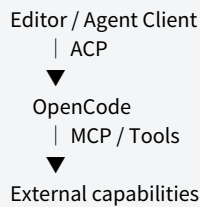
import { createOpenCode } from "@opencode-ai/sdk"
const { client } = await createOpenCode()
```

官方 SDK 是 OpenCode Server 的类型安全 JS/TS 客户端，并支持 structured output（JSON schema）。

18.4 ACP

```
opencode acp
```

ACP 模式将 OpenCode 作为 ACP-compatible subprocess，通过 stdio + JSON-RPC 与编辑器通信。



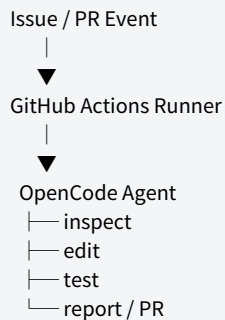
概念区分

ACP 解决 “Editor ↔ Agent”；MCP 解决 “Agent ↔ Tool/Service”。不要把两者混为一谈。

19. GitHub Automation：从交互 Agent 到软件工程流水线

```
opencode github install
```

官方安装流程会帮助安装 GitHub App、创建 workflow、设置 secrets。OpenCode GitHub Action 可由 issue/PR comment 中的 `/oc` 或 `/opencode` 触发，也支持 issue/PR 事件自动化。



19.1 适合自动化的任务

- 只读 PR review。

- issue 初步归类与上下文汇总。
- TODO / deprecation 扫描。
- 生成测试建议。
- 在严格 Permission/CI 下执行小范围修复。

19.2 不要一开始全自动

推荐成熟度阶梯

第 1 阶段只读分析 → 第 2 阶段生成 patch 但人工合并 → 第 3 阶段自动开 PR → 第 4 阶段仅对低风险规则化变更自动合并。每升一级都要求评测数据、权限边界和回滚能力。

20. 12 个高频 Recipe

Recipe 1 - 接手陌生仓库

1. 运行 `/init` 或先审查现有 AGENTS.md。
2. 切 Plan：让 Agent 画模块图、入口、关键数据流、测试路径。
3. 用 `@` 精确附加 README/architecture/package config。
4. 让 Explore 子代理分别搜索入口、测试、配置。
5. 输出“已确认事实 / 未确认假设 / 下一步验证”三栏。

推荐 Prompt

```
Explore this repository without modifying files.
Return:
1. architecture map
2. main entry points
3. build/test/lint commands
4. top 5 risky areas
5. unknowns that require verification
```

Recipe 2 - 修复 Bug

6. 先复现，不先改。
7. 收集最小执行路径与失败证据。
8. 提出 1-3 个根因假设并逐一证伪。
9. 选最小 patch。
10. 先 focused test，再全局相关验证。

推荐 Prompt

```
Investigate the reported bug.
Do not edit files until the failure is reproduced or the root cause is supported by evidence.
Then propose the smallest safe fix and the exact verification commands.
```

Recipe 3 - 实现新功能

11. Plan 阶段列 API/数据模型/兼容性/测试面。
12. 检查已有抽象，避免重复造轮子。

13. 将任务切为可独立验证的小 patch。
14. 每个 patch 后跑 focused validation。

推荐 Prompt

Plan the feature before coding.
Identify existing patterns to reuse, API compatibility constraints, affected tests, and a sequence of independently verifiable changes.

Recipe 4 - 大型 Refactor

15. 明确“不改变行为”的 invariant。
16. 先建立或补齐 characterization tests。
17. 按模块/接口逐步迁移。
18. 每一步都保持可运行。
19. 最后使用 Reviewer 子代理看回归风险。

推荐 Prompt

Refactor without changing observable behavior.
First identify invariants and existing tests.
Propose a sequence of small reversible steps.
After each step run focused verification.

Recipe 5 - Code Review

20. Reviewer 设 edit=deny。
21. 输入 git diff 而非整个仓库。
22. 要求区分 blocker / risk / nit。
23. 每条问题给出证据位置与触发条件。

推荐 Prompt

Review the current diff only.
Classify findings as blocker, risk, or nit.
For each finding include evidence, impact, and a minimal remediation.
Do not edit files.

Recipe 6 - 补测试

24. 先识别行为边界和失败模式。
25. 优先覆盖 bug regression 与 contract。
26. 避免只测实现细节。
27. 验证测试在修复前会失败、修复后通过。

推荐 Prompt

Add tests for the behavior, not implementation details.
Prioritize regression coverage, boundary cases, and public contracts.
Explain what each test protects.

Recipe 7 - 依赖升级

28. Scout 查上游 changelog/源码。
29. 明确 breaking changes。
30. 先升级锁文件/依赖，再逐点修复。
31. 运行全套与依赖相关验证。

推荐 Prompt

Research the dependency upgrade first.
Identify breaking changes that affect this repository, then propose the smallest migration sequence.

Recipe 8 - 数据库 Migration

32. 使用专用 Skill。
33. 检查 prior migrations 与兼容窗口。
34. 生成后读 SQL。
35. 测试迁移与回退/forward-fix。
36. 不允许连接生产环境。

推荐 Prompt

Use the database-migration skill.
Treat backwards compatibility and rollback/forward-fix as first-class requirements.

Recipe 9 - 多仓库对照实现

37. 将 SDK/上游作为 Reference。
38. 主 repo 只做当前实现。
39. 让 Agent 明确引用外部文件证据。
40. 不要无差别复制上游代码。

推荐 Prompt

Compare the local implementation with @sdk.
Explain semantic differences first; only then propose changes.

Recipe 10 - 本地模型实验

41. 使用 OpenAI-compatible provider。
42. 配置 context/output limits。
43. 先测工具选择正确率。
44. 再测多步任务与长上下文。
45. 记录成功率、token、步骤数。

推荐 Prompt

Run a small benchmark: repository Q&A, one-file edit, multi-file bugfix.
Record tool-choice errors and verification failures separately from model text quality.

Recipe 11 - CLI 批处理

- 46. 常驻 `opencode serve`。
- 47. 每个任务独立 session。
- 48. 输出结构化 JSON（SDK structured output 更稳定）。
- 49. 对失败任务保留 session 以复盘。

推荐 Prompt

For batch jobs, separate task execution from result aggregation.
Keep one session per task and return structured result fields.

Recipe 12 - CI / GitHub Review Bot

- 50. 先只读。
- 51. Agent 不能拥有超过任务需要的 token 权限。
- 52. 所有修改通过 branch/PR。
- 53. 以 CI 为最终 gate。
- 54. 持续统计误报和人工接管率。

推荐 Prompt

Review the PR as a read-only agent.
Only report findings with concrete evidence.
Do not modify code or perform external writes.

21. 团队治理与 Agent Eval

真正进入团队使用后，最重要的问题从“Agent 能不能做”变成“它在什么边界内稳定做到、成本是多少、失败如何被发现”。

21.1 建议评测指标

维度	指标示例	为什么重要
任务质量	task success / tests pass	最终结果
工具行为	invalid tool call / wrong tool / redundant calls	Harness 质量
效率	tokens / steps / wall time	成本与延迟
人机协作	approval count / takeover rate	操作负担
安全	denied-action attempts / external writes	边界可靠性
回归	post-merge incidents / reverted PRs	真实工程后果

21.2 评测分层

L0 Tool unit eval
→ 能否选对工具、参数是否正确
L1 Repository task eval
→ 小 bug / test / refactor
L2 Workflow eval
→ Plan → Build → Verify
L3 Production telemetry
→ 人工接管、PR 回滚、成本、事故

可累积 know-how

每次 Agent 失败都先分类：Model knowledge、Context missing、Tool affordance、Permission、Runtime、Verification。只有分类后，才能知道应该改 prompt、AGENTS、Skill、Tool schema，还是模型。

22. Troubleshooting

Provider 连接了但模型不可用

运行 `opencode auth list`；检查 provider ID 是否一致；检查 baseURL 与 AI SDK package；确认 model ID 为 provider/model。

Agent 总在乱搜

在 Prompt/AGENTS 指定入口与验证命令；使用 @file；创建 Explore/Scout 分工；降低工具/Reference 暴露。

Skill 不出现

确认文件名必须是 `SKILL.md`；frontmatter 有 name/description；name 与目录匹配且唯一；检查 skill permission 是否 deny。

MCP 工具太多导致上下文爆炸

只启用任务需要的 MCP；按 Agent 限制工具；将低频能力移出默认运行集。

Permission 规则“没按预期工作”

记住最后匹配规则获胜；把 `\*` 放前面，具体规则放后面；用最小命令模式测试。

Plan 还是修改了文件

检查是否覆盖了内置 Plan 的 permission；自定义 Plan 明确 `edit: deny`；不要只靠 prompt 说“不要改”。

Auto 模式风险太高

先写明确 deny，再开 --auto；Auto 不应成为绕过治理的方式。

长会话开始重复/遗忘

运行 `/compact`；把稳定结论写入 AGENTS/Skill；新阶段考虑新 session。

/undo 不工作

官方 TUI 文档指出文件回滚依赖 Git；确认项目是 Git repo 且不要混入未追踪的重要本地状态。

LSP 没诊断

确认 `lsp` 已启用且 server 可启动；最终仍以项目 compiler/typecheck/test 为准。

Server 无法远程连接

确认 hostname/port/CORS；若跨主机暴露，配置 SERVER password 并加网络层保护。

Tool 会被误用

缩窄 tool description、参数 schema、枚举；减少重叠工具；增加 eval；必要时 permission deny/ask。

23. 可沉淀的 OpenCode know-how

1. 模型不是 Agent

Coding Agent 的结果来自模型、上下文、工具、权限、Runtime、验证共同作用。

2. Context Window 是工作内存

只把“此刻决定下一步”需要的信息放进去。

3. AGENTS.md 是 Repository Contract

保存稳定、高频、全局知识，而不是所有文档。

4. Skill 是程序化组织经验

将低频但复杂的 SOP 做成 on-demand instruction。

5. Command 是显式工作流入口

把反复复制的 Prompt 变成可版本化接口。

6. Tool Schema 是 Prompt

给概率性调用者设计 affordance，而不仅是类型正确。

7. Permission 是 Action Space

deny/allow 不只是 UX，也塑造模型能考虑的动作集合。

8. Subagent 的核心是 Context Isolation

并行只是副产品；隔离中间噪声更关键。

9. MCP 不是越多越好

工具 schema 也消耗上下文；最小可用工具集更稳定。

10. Autonomy 必须配 Reversibility

Git、undo、PR、checkpoint 降低失败成本。

11. Verification 必须外部化

测试、编译器、CI、diff 比模型自我声明更可信。

12. Agent 失败要做归因

先定位失败层，再优化；不要每次都只换模型。

附录 A. 配置模板

A1. 保守个人开发配置

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true,
  "permission": {
    "**": "ask",
    "read": "allow",
    "glob": "allow",
    "grep": "allow",
    "lsp": "allow",
    "skill": "allow",
    "bash": {
      "**": "ask",
      "git status*": "allow",
      "git diff*": "allow",
      "git log*": "allow",
      "git push*": "deny"
    }
  },
  "compaction": {"auto": true, "prune": true, "reserved": 10000},
  "watcher": {"ignore": ["node_modules/**", "dist/**", ".git/**"]}
}
```

A2. 只读 Review 环境

```
{
  "permission": {
    "**": "deny",
    "read": "allow",
    "glob": "allow",
    "grep": "allow",
    "lsp": "allow",
    "bash": {
      "**": "deny",
      "git status*": "allow",
      "git diff*": "allow",
      "git log*": "allow"
    }
  }
}
```

A3. Team Agents

```
{
  "agent": {
    "reviewer": {
      "description": "Review changes without editing files",
      "mode": "subagent",
      "temperature": 0.1,
      "permission": {
        "edit": "deny",
        "bash": {"*": "ask", "git diff*": "allow"}
      }
    },
    "tester": {
      "description": "Run focused tests and explain failures",
      "mode": "subagent",
      "permission": {
        "edit": "deny",
        "bash": "ask"
      }
    }
  }
}
```

附录 B. Agent / Skill / Command 模板

B1. Explorer

```
---
description: Explore code paths and repository structure without modifications
mode: subagent
temperature: 0.1
permission:
  edit: deny
  bash: ask
---
Return evidence with file paths. Separate confirmed facts from hypotheses.
```

B2. Security-minded Reviewer

```
---
description: Review code changes for correctness, security and regression risk
mode: subagent
temperature: 0.1
permission:
  edit: deny
---
Review only. For each issue provide evidence, impact, trigger conditions and remediation.
```

B3. Release Skill

```
---
name: release
description: Prepare a release using repository versioning and changelog conventions
---
1. Read release docs and recent tags.
2. Summarize merged changes.
3. Propose version bump with rationale.
4. Run release verification commands.
5. Produce release notes.
Never publish without explicit user approval.
```

B4. Investigate Command

```
---
description: Investigate a bug before editing
agent: plan
---
Investigate: $ARGUMENTS

Do not edit files.
Return reproduction, execution path, evidence, root-cause hypotheses, and verification plan.
```

附录 C. 命令速查

命令	用途
opencode	启动 TUI
opencode [project]	指定项目启动
/connect	连接 provider / credential
/models	选择模型
/init	生成/更新 AGENTS.md
/compact	压缩当前 session
/undo	撤销最近消息和对应文件改动
opencode run "..."	非交互运行
opencode serve	启动 headless server
opencode mcp list	查看 MCP 状态
opencode mcp auth <name>	MCP OAuth
opencode agent list	查看 agents
opencode github install	安装 GitHub workflow
opencode acp	启动 ACP subprocess
opencode stats	统计使用情况

附录 D. 官方资料索引与版本说明

本手册的产品事实以 OpenCode 官方文档与 anomalyco/opencode 仓库为基准，资料检索/核验日期为 2026-08-13。

OpenCode 更新频繁，具体字段在未来版本可能变化；遇到冲突时以当前 `https://opencode.ai/docs` 与配置 schema 为准。

- [S1] OpenCode GitHub repository — <https://github.com/anomalyco/opencode>
- [S2] Intro — <https://opencode.ai/docs/>
- [S3] Config — <https://opencode.ai/docs/config/>
- [S4] Providers — <https://opencode.ai/docs/providers/>
- [S5] Rules / AGENTS.md — <https://opencode.ai/docs/rules/>
- [S6] Agents — <https://opencode.ai/docs/agents/>
- [S7] Permissions — <https://opencode.ai/docs/permissions/>
- [S8] Agent Skills — <https://opencode.ai/docs/skills/>
- [S9] Commands — <https://opencode.ai/docs/commands/>
- [S10] Custom Tools — <https://opencode.ai/docs/custom-tools/>
- [S11] MCP Servers — <https://opencode.ai/docs/mcp-servers/>
- [S12] Plugins — <https://opencode.ai/docs/plugins/>
- [S13] TUI — <https://opencode.ai/docs/tui/>
- [S14] CLI — <https://opencode.ai/docs/cli/>
- [S15] Server — <https://opencode.ai/docs/server/>
- [S16] SDK — <https://opencode.ai/docs/sdk/>
- [S17] References — <https://opencode.ai/docs/references/>
- [S18] LSP — <https://opencode.ai/docs/lsp/>
- [S19] ACP — <https://opencode.ai/docs/acp/>
- [S20] GitHub integration — <https://opencode.ai/docs/github/>
- [S21] Security / threat model — <https://github.com/anomalyco/opencode/security>

关于“推荐配置”

手册中标注为“推荐”“基线”“成熟度阶梯”的内容属于基于官方机制推导出的工程实践，不是 OpenCode 官方强制规范；产品事实与配置语义来自上方官方资料。